13.3. Insertion and Deletion in Graphs

Previous Section:

 13.2. Graphs Traversal

Contents:

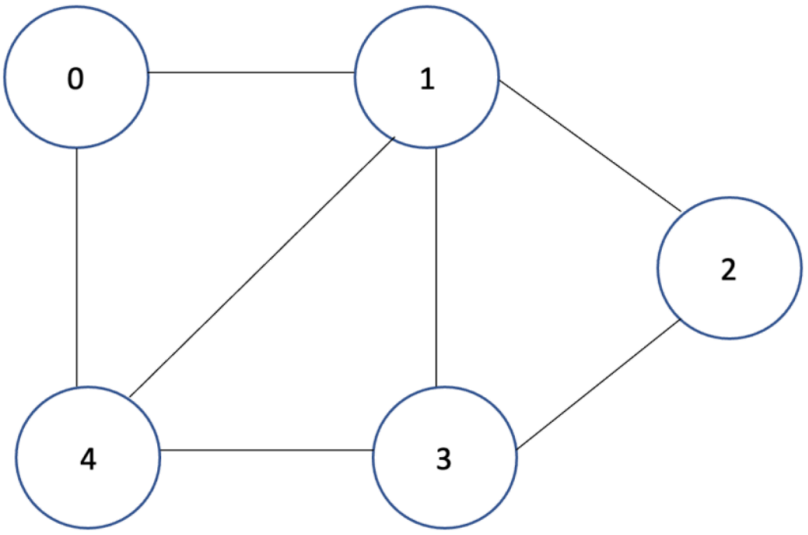
Like any other data structure, graphs can also be modified after they have been created. We may insert or delete **edges** to change the relationships between vertices, or insert or delete **vertices** to expand or shrink the graph.

The exact implementation depends on whether the graph is represented using an **Adjacency List** or an **Adjacency Matrix**, but the underlying concepts remain the same.

1. Insertion and Deletion of Edges

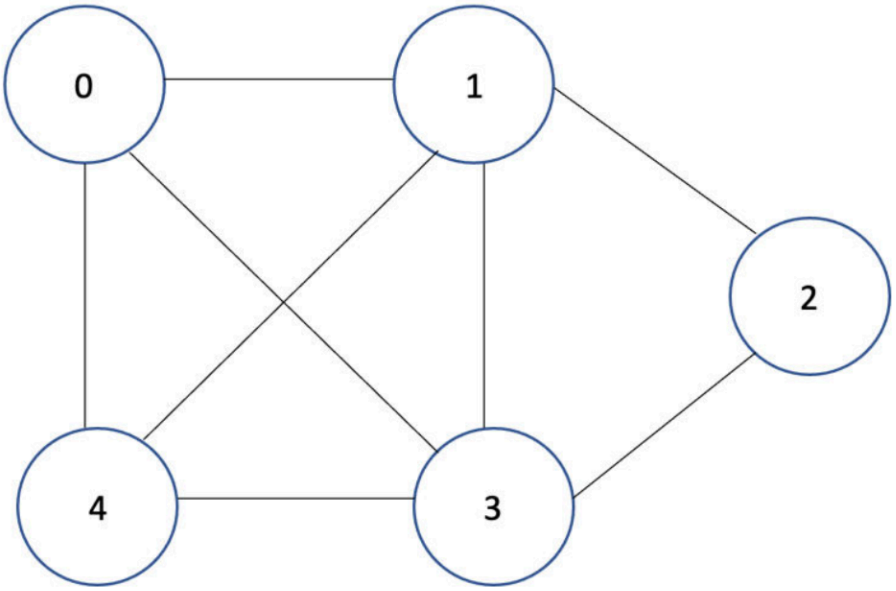
An **edge** represents a connection between two vertices. Since no vertices are added or removed, edge operations are relatively straightforward.

Consider the following undirected graph represented as an adjacency list.




```
Adjacency List
0 : 1 4
1 : 0 2 3 4
2 : 1 3
3 : 1 2 4
4 : 0 1 3
```

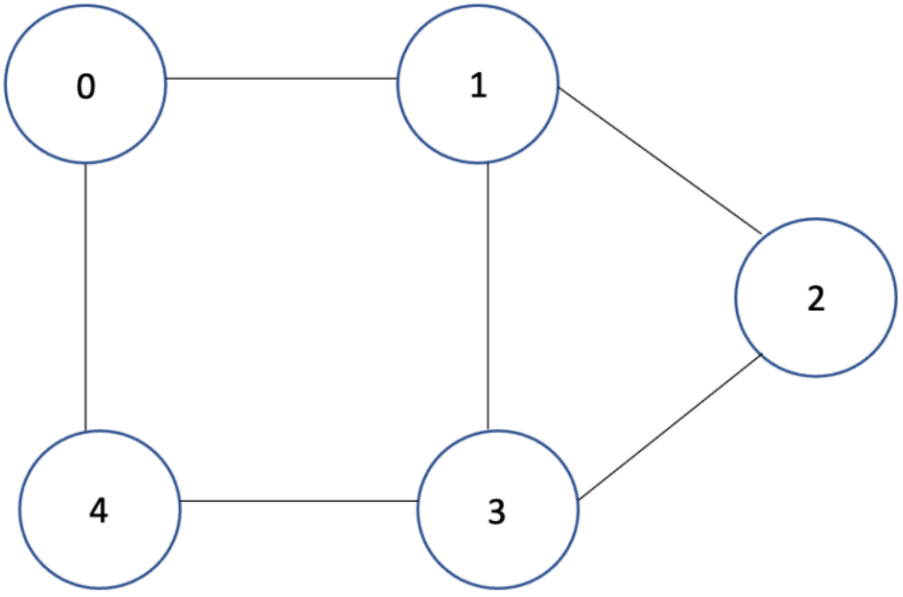
- **Inserting an Edge:** Suppose we insert a new edge between **vertex 0** and **vertex 3**.



```
Updated Adjacency List
0 : 1 3 4
1 : 0 2 3 4
2 : 1 3
3 : 0 1 2 4
4 : 0 1 3
```

The newly inserted edge `(0,3)` creates an additional connection between the two vertices.

- Deleting an Edge: Now return to the **original graph** and remove the edge between **vertex 1** and **vertex 4**.




```
Updated Adjacency List
0 : 1 4
1 : 0 2 3
2 : 1 3
3 : 1 2 4
4 : 0 3
```

Notice that only the relationship between vertices **1** and **4** is removed. The remaining graph is unchanged.

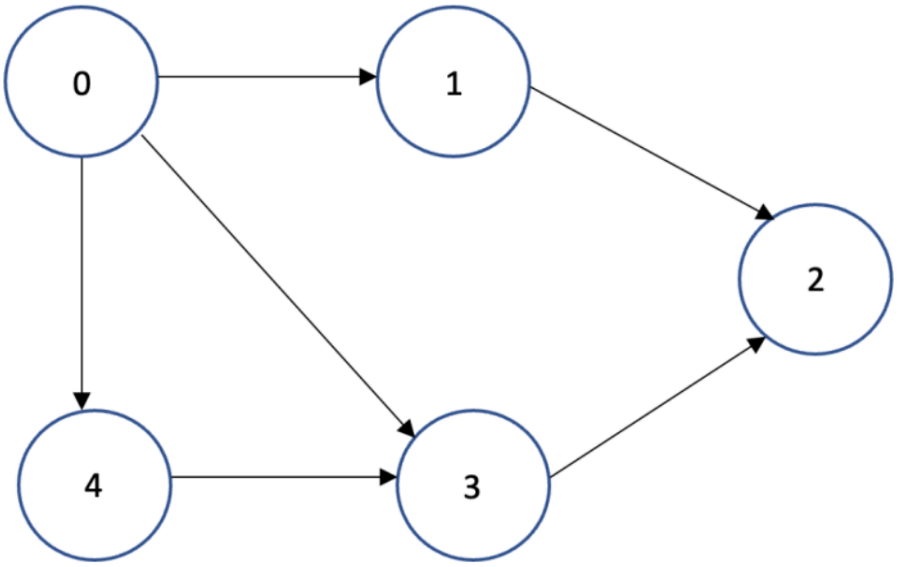
- Python Implementation: Assume the `Graph` class introduced in the previous section stores an adjacency matrix in `self.matrix`.

```
import numpy as np

class Graph:

    def __init__(self, vertices):
        self.V = vertices
        self.matrix = np.zeros((vertices, vertices), dtype=
int)

    def insert_edge(self, u, v):
        """Insert an edge into an undirected graph."""
        self.matrix[u][v] = 1
        self.matrix[v][u] = 1

    def delete_edge(self, u, v):
        """Delete an edge from an undirected graph."""
        self.matrix[u][v] = 0
        self.matrix[v][u] = 0
```

2. Insertion and Deletion of Vertices

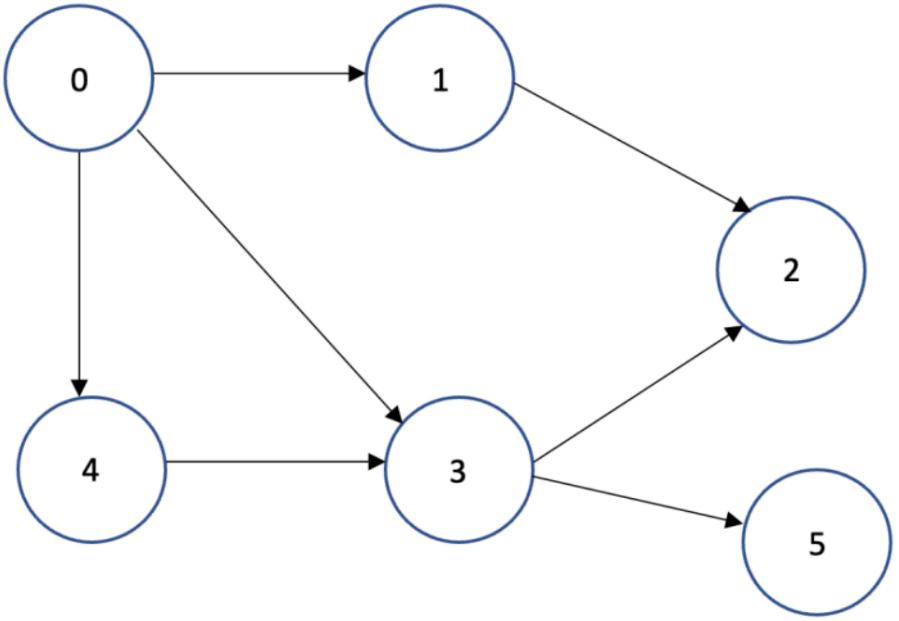
Unlike edge operations, inserting or deleting a vertex changes the size of the graph.

When a new vertex is inserted, the graph gains an additional row and column in the adjacency matrix (or a new list in the adjacency list).




```
Adjacency List
0 : 1 3 4
1 : 2
2 :
3 : 2
4 : 3
```

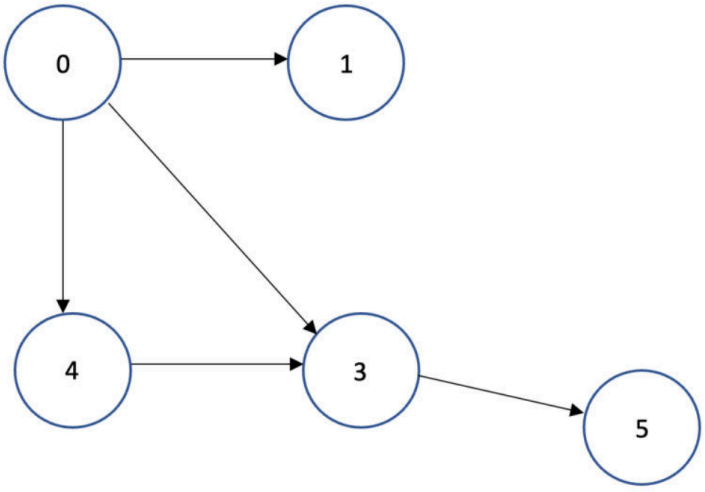
- Inserting a Vertex: Insert **vertex 5**, making it adjacent to **vertex 3**.



```
Updated Adjacency List
0 : 1 3 4
1 : 2
2 :
3 : 2 5
4 : 3
5 : 3
```

The graph now contains one additional vertex and one additional edge.

- Deleting a Vertex: Now delete **vertex 2** from the previous graph. Deleting a vertex removes the vertex itself and every edge connected to it




```
Updated Adjacency List
0 : 1 3 4
1 :
3 : 5
4 : 3
5 : 3
```

Notice that every edge connected to vertex **2** disappears automatically.

Note: In an undirected graph, inserting a new vertex connected to an existing vertex requires updating **both adjacency lists**. For example, if

vertex **5** is connected to vertex **3**, then:

- Vertex **3** stores **5** as a neighbor.
- Vertex **5** stores **3** as a neighbor.

- Python Implementation

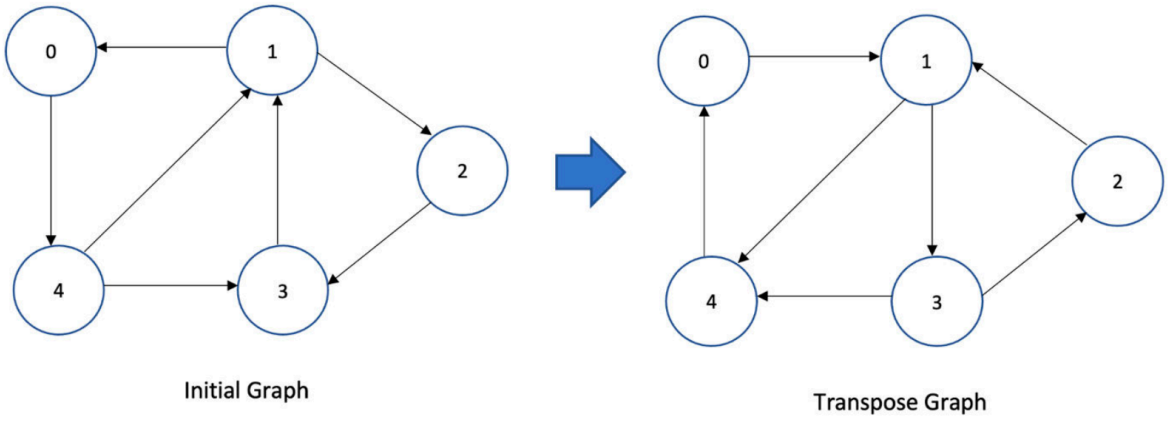
```
import numpy as np

class Graph:

    def __init__(self, vertices):
        self.V = vertices
        self.matrix = np.zeros((vertices, vertices), dtype=
int)

    def insert_vertex(self, position=None):
        """
        Insert a new isolated vertex.

        If position is None, append the vertex
        to the end of the adjacency matrix.
        """

        if position is None:
            position = self.V

        self.matrix = np.insert(self.matrix, position, 0, a
xis=0)
        self.matrix = np.insert(self.matrix, position, 0, a
xis=1)

        self.V += 1

    def delete_vertex(self, vertex):
        """Delete a vertex from the graph."""
```

```
        self.matrix = np.delete(self.matrix, vertex, axis=
0)
        self.matrix = np.delete(self.matrix, vertex, axis=
1)

        self.V -= 1
```

3. Transpose of a Graph

The **transpose** of a graph is obtained by **reversing the direction of every edge**.

- For **directed graphs**, every edge `(u → v)` becomes `(v → u)`.
- For **undirected graphs**, the transpose is identical to the original graph because every edge already exists in both directions.



For an adjacency matrix, the transpose graph is simply the **matrix transpose**.

- Python Implementation

```
import numpy as np

class Graph:

    def __init__(self, vertices):
        self.V = vertices
```

```
        self.matrix = np.zeros((vertices, vertices), dtype=
int)

    def transpose(self):
        """Return the transpose of the graph."""
        return self.matrix.T.copy()
```

Using NumPy's built-in transpose (`.T`) is both simpler and more efficient than manually swapping every matrix element. It produces the same result while requiring only a single statement.

Summary

In this section, we learned how graphs can be modified by inserting and deleting both edges and vertices.

- We saw that edge operations simply update the connections between existing vertices, while vertex operations change the size of the graph by adding or removing entire rows and columns in the adjacency matrix.
- We also explored graph transposition, where the direction of every edge in a directed graph is reversed by transposing its adjacency matrix.

Finally, these operations were implemented as methods of a `Graph` class, providing a clean and object-oriented approach to managing and modifying graph structures.

References

https://drive.google.com/file/d/1R4R6eGz_XZEva12XTJEv3olhZt1aet4x/view?usp=drive_link

https://www.includehelp.com/ds/insertion-and-deletion-of-nodes-and-edges-in-a-graph-using-adjacency-list.aspx

Next Section:

14. Sorting Algorithms